



DC – AI – Routing – Switching – Security – Wireless – Automation

Operating MX/SRX Scale-Out System - Bulk Junos Config Changes

An HPE Juniper Networking Techpost

Karel Hendrych

2024-02-22

HPE



Contents

1	Operating MX/SRX Scale-Out System - Bulk Junos Config Changes	3
1.1	Introduction	3
1.2	The challenge	3
1.3	The tool	4
1.4	Troubleshooting	8
1.5	Tool Help	9
1.6	Conclusion	10
1.6.1	Availability	10
1.7	Useful links	10
1.8	Glossary	10
1.9	Acknowledgments	11

List of Figures

1 **Topology** 3

1 Operating MX/SRX Scale-Out System - Bulk Junos Config Changes

Karel Hendrych - 02/22/2024

A minimalistic tool for bulk config changes in the scale-out system beyond options available in Auto-FBF CLI

1.1 Introduction

This TechPost is continuation of “[Scale-Out Security Services with Auto-FBF](#) looking at the things more practically. Now the focus will be on a minimalistic tool for bulk config changes in the scale-out system beyond options available in Auto-FBF CLI. To a certain extent the tool could be used as a very simplistic alternative to templating features of Security Director and other external tooling. In this specific example the tool is used for controlling configuration of scaled-out SRX firewalls from MX/PTX device. Reading previous TechPosts is recommended for understanding the Auto-FBF principles. However, approaches are applicable elsewhere too due to generic nature for any Junos devices.

1.2 The challenge

Let's imagine a simple topology from previous TechPost consisting of Juniper Networks [MX304 router](#) as the scale-out distribution device and four [SRX4600 firewalls](#).

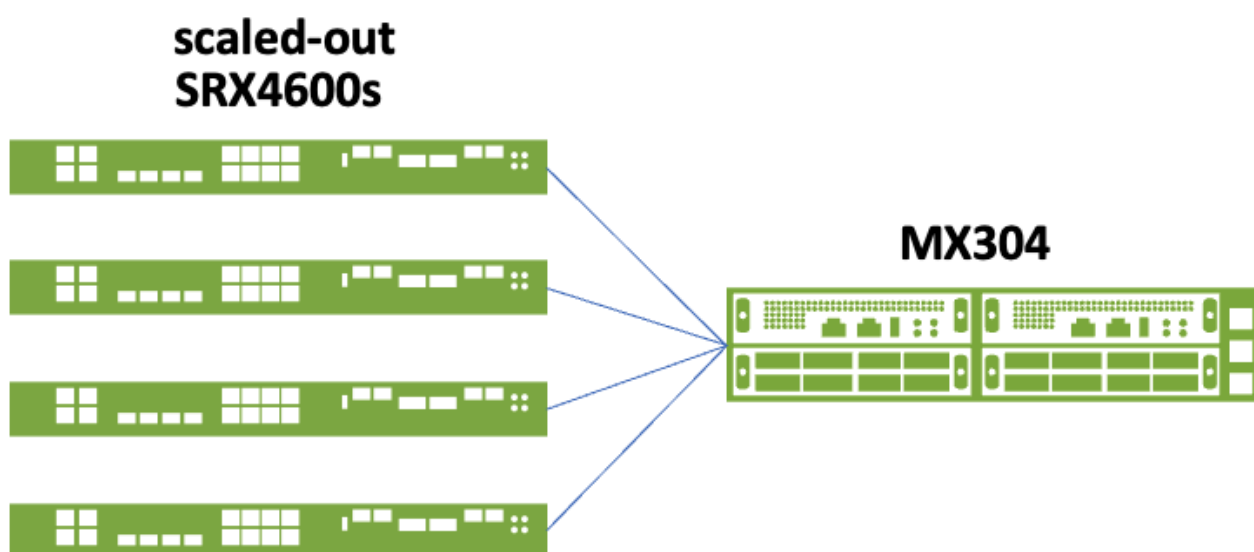


Figure 1: Topology

Configuration changes done manually to all four devices (possibly many more) are impractical and prone to errors. Therefore, flexible Python based tool for on-box operation with following features:

- Conduct Junos config changes from Auto-FBF distribution device(s) like the MX/PTX (or any other Junos / PyEZ equipped machine)
- Use Jinja2 templates
- Have optional workflow using prototype device for validation and verification prior bulk change
- Preview Junos set commands for prototype device

- Do diff of running/candidate config on prototype device
- Load configuration to prototype device
- Able to operate on multiple devices using sequence modifier. For example, the devices having sequentially lined up interface IPs, next /30 prefix, BGP ASNs, etc.
- Have a profile driven bulk configuration push
- Profiles able to mix different device types with different templates, e.g., vSRX/SRX4600
- Control what device types are eligible for configuration push and/or only for viewing rendered set commands from template (safety and operator review prior Junos load set)
- Easy to expand by new template variables inside of the code for given device type, simple expansion for template data from external data sources
- Thorough logging, including template rendering debug
- Ability to archive committed templates and set commands for audit and roll-back purposes
- General purpose where the tool could be de-coupled from Auto-FBF; or placed elsewhere than MX/PTX to manage configuration (any PyEZ library equipped machine).

The idea of prototype prior changing everything comes from real-life when changes can cause unforeseen consequences. One or more devices in the scale-out system can be designated for testing changes including longevity test prior roll out to whole system. Besides that, Auto-FBF has concept of groups where one or more devices can be designated for various purposes, one of them can be prototype device(s) group serving limited number of subscribers (e.g., mobile SP testing APN). Generally, this is one of the greatest operational advantages of scale-out systems that changes can be thoroughly tested including real-life traffic patterns on smaller set of devices.

1.3 The tool

First step is to list preconfigured templates stored in predefined folder on the MX304, MD5 is handy for tracing in logs what template got pushed where and whether the template has been changed:

```

1 root@mx304-20-re0> op device-gen-auto-fbf list-template all
2 -----
3 | # | /var/db/scripts/op/template/*.j2 | md5 |
4 -----
5 | 1 | 01-mx304-20-srx4600 | 5192e37ebbc55f5fbfd3fe921da5f9c2 |
6 -----
7 | 2 | 01-srx4600-all | 3b2b986b1ad8b4d5bab3740f412953b9 |
8 -----
9 | 3 | 01-srx4600-common | 153f5f6f81574567633a75e691c95f7c |
10 -----
11 | 4 | 01-srx4600-local | 9d6514d63c11753927e6b9801cdccd4e |
12 -----

```

To show contents of a template defining SRX4600 specific settings - CLI show command is available as well:

```

1 root@mx304-20-re0> op device-gen-auto-fbf show-template 01-srx4600-local
2 #
3 ##### BEGIN 01-srx4600-local #####
4 #
5 delete groups auto-fbf-local
6 delete groups auto-fbf
7
8 set groups auto-fbf-local security log stream server-35 source-address 100.65.0.{{ seq_p1 }}
9 set groups auto-fbf-local interfaces ae0 unit 1000 family inet address 100.64.0.{{ seq_p1 }}/24
10 set groups auto-fbf-local interfaces ae0 unit 1000 family inet6 address dead:64::{{ seq_p1 }}/64
11 set groups auto-fbf-local interfaces ae0 unit 1001 family inet address 100.65.0.{{ seq_p1 }}/24
12 set groups auto-fbf-local interfaces ae0 unit 1001 family inet6 address dead:65::{{ seq_p1 }}/64
13 set groups auto-fbf-local routing-instances vr-gi routing-options autonomous-system {{ aut_sys
14     }}

```

```

15 set groups auto-fbf security nat source pool pool-1 address 3.0.{{ seq }}.0/24
16 set groups auto-fbf routing-instances vr-gi routing-options static route 3.0.{{ seq }}.0/24
   discard
17
18 set groups auto-fbf-local chassis aggregated-devices ethernet device-count 1
19 set groups auto-fbf-local security zones security-zone inet interfaces ae0.1000
20 set groups auto-fbf-local security zones security-zone gi interfaces ae0.1001
21 set groups auto-fbf-local interfaces ae0 vlan-tagging
22 set groups auto-fbf-local interfaces ae0 aggregated-ether-options lacp active
23 set groups auto-fbf-local interfaces ae0 aggregated-ether-options lacp periodic fast
24 set groups auto-fbf-local interfaces ae0 unit 1000 description inet
25 set groups auto-fbf-local interfaces ae0 unit 1000 vlan-id 1000
26 set groups auto-fbf-local interfaces ae0 unit 1001 description gi
27 set groups auto-fbf-local interfaces ae0 unit 1001 vlan-id 1001
28 set groups auto-fbf-local interfaces et-1/0/0 ether-options 802.3ad ae0
29 set groups auto-fbf-local interfaces et-1/0/1 ether-options 802.3ad ae0
30 set groups auto-fbf-local interfaces et-1/0/2 ether-options 802.3ad ae0
31 set groups auto-fbf-local interfaces et-1/0/3 ether-options 802.3ad ae0
32 set groups auto-fbf-local routing-instances vr-gi interface ae0.1000
33 set groups auto-fbf-local routing-instances vr-gi interface ae0.1001
34 set groups auto-fbf-local routing-instances vr-gi protocols bgp bfd-liveness-detection minimum-
   interval 300
35 set groups auto-fbf-local routing-instances vr-gi protocols bgp bfd-liveness-detection
   multiplier 3
36
37 set apply-groups auto-fbf-local
38 set apply-groups auto-fbf
39 #
40 ##### END 01-srx4600-local
   #####
41 #

```

The above template has variable for sequentially increased parameter (seq) and by +1 (seq_p1) for bulk operations. Junos groups are used to ensure starting over during every template operation by deleting the groups at the beginning.

To preview Junos set commands rendered from the given template for defined device type (setting variables for template) and for specific sequence:

```

1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local seq 7
2 delete groups auto-fbf-local
3 delete groups auto-fbf
4 set groups auto-fbf-local interfaces ae0 unit 1000 family inet address 100.64.0.8/24
5 set groups auto-fbf-local interfaces ae0 unit 1000 family inet6 address dead::8/64
6 set groups auto-fbf-local interfaces ae0 unit 1001 family inet address 100.65.0.8/24
7 set groups auto-fbf-local interfaces ae0 unit 1001 family inet6 address dead::8/64
8 set groups auto-fbf-local routing-instances vr-gi routing-options autonomous-system 65207
9
10 set groups auto-fbf security nat source pool pool-1 address 3.0.7.0/25
11 set groups auto-fbf routing-instances vr-gi routing-options static route 3.0.7.0/25 discard
12
13 set groups auto-fbf-local chassis aggregated-devices ethernet device-count 1
14 set groups auto-fbf-local security zones security-zone inet interfaces ae0.1000
15 set groups auto-fbf-local security zones security-zone gi interfaces ae0.1001
16 set groups auto-fbf-local interfaces ae0 vlan-tagging
17 set groups auto-fbf-local interfaces ae0 aggregated-ether-options lacp active
18 set groups auto-fbf-local interfaces ae0 aggregated-ether-options lacp periodic fast
19 set groups auto-fbf-local interfaces ae0 unit 1000 description inet
20 set groups auto-fbf-local interfaces ae0 unit 1000 vlan-id 1000
21 set groups auto-fbf-local interfaces ae0 unit 1001 description gi
22 set groups auto-fbf-local interfaces ae0 unit 1001 vlan-id 1001
23 set groups auto-fbf-local interfaces et-1/0/0 ether-options 802.3ad ae0
24 set groups auto-fbf-local interfaces et-1/0/1 ether-options 802.3ad ae0
25 set groups auto-fbf-local interfaces et-1/0/2 ether-options 802.3ad ae0
26 set groups auto-fbf-local interfaces et-1/0/3 ether-options 802.3ad ae0
27 set groups auto-fbf-local routing-instances vr-gi interface ae0.1000
28 set groups auto-fbf-local routing-instances vr-gi interface ae0.1001
29 set groups auto-fbf-local routing-instances vr-gi protocols bgp bfd-liveness-detection minimum-
   interval 300
30 set groups auto-fbf-local routing-instances vr-gi protocols bgp bfd-liveness-detection
   multiplier 3
31
32 set apply-groups auto-fbf-local
33 set apply-groups auto-fbf

```

To view diff between running and candidate configuration from given template on a prototype device (selected

was srx-07 – also matching sequence 7), for example when SRX template above would increase NAT pool size from /26 to /25:

```
1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local diff-
  target srx-07 seq 7
2 [edit groups auto-fbf security nat source pool pool-1 address]
3 + 3.0.7.0/25;
4 - 3.0.7.0/26;
5 [edit groups auto-fbf routing-instances vr-gi routing-options static]
6 + route 3.0.7.0/25 discard;
7 - route 3.0.7.0/26 discard;
```

The diff is working on rollback 0 - the recommended approach are junos groups avoiding risk of changing other group(s) than in template and core configuration for remote out of band management outside of groups.

Also, diff is handy to see difference between template modified by sequence applied to non-matching device - effectively showing difference between devices. For example, sequence 7 applied to srx-08 (would have been normally srx-07 as in previous example):

```
1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local diff-
  target srx-08 seq 7
2 [edit groups auto-fbf-local security log stream server-35]
3 - source-address 100.65.0.9;
4 + source-address 100.65.0.8;
5 [edit groups auto-fbf-local interfaces ae0 unit 1000 family inet]
6 + address 100.64.0.8/24;
7 - address 100.64.0.9/24;
8 [edit groups auto-fbf-local interfaces ae0 unit 1000 family inet6]
9 + address dead:64::8/64;
10 - address dead:64::9/64;
11 [edit groups auto-fbf-local interfaces ae0 unit 1001 family inet]
12 + address 100.65.0.8/24;
13 - address 100.65.0.9/24;
14 [edit groups auto-fbf-local interfaces ae0 unit 1001 family inet6]
15 + address dead:65::8/64;
16 - address dead:65::9/64;
17 [edit groups auto-fbf-local routing-instances vr-gi routing-options]
18 - autonomous-system 65208;
19 + autonomous-system 65207;
20 [edit groups auto-fbf security nat source pool pool-1 address]
21 + 3.0.7.0/26;
22 - 3.0.8.0/25;
23 [edit groups auto-fbf routing-instances vr-gi routing-options static]
24 + route 3.0.7.0/26 discard;
25 - route 3.0.8.0/25 discard;
```

Next step is configuration push to the prototype device:

```
1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local push-
  target srx-07 seq 7
2 -----
3 | device | template operation status message |
4 -----
5 | srx-07 | 01-srx4600-local template commit complete |
6 -----
```

Change made to prototype device srx-07 seen using Auto-FBF nat-info tool:

```
1 root@mx304-20-re0> op auto-fbf srx-nat-info all
2 -----
3 | instance | pool name | range | IPs | usage | peak usage | peak usage time | % failed session |
4 -----
5 | srx-07 | pool-1 | 3.0.7.0 - 3.0.7.127 | 128 | 0% | 0% | 1970-01-01 01:00:00 CET | 0.000 |
6 -----
7 | srx-08 | pool-1 | 3.0.8.0 - 3.0.8.63 | 64 | 0% | 77% | 2023-12-29 02:00:30 CET | 0.000 |
8 -----
9 | srx-09 | pool-1 | 3.0.9.0 - 3.0.9.63 | 64 | 0% | 75% | 2023-12-29 02:00:30 CET | 0.000 |
```

```

10 -----
11 | srx-10 | pool-1 | 3.0.10.0 - 3.0.10.63 | 64 | 0% | 76% | 2023-12-29 02:00:30 CET | 0.000 |
12 -----

```

And corresponding log records about push and commit of specific template to srx-07:

```

1 [root]/var/db/scripts/op/auto-fbf-gen-dev.py[99906] srx-07 template push /var/db/scripts/op/
  template/01-srx4600-local.j2 (md5: d31be14dadd2ce3437b2c1a3d84fd39d)
2 [root]/var/db/scripts/op/auto-fbf-gen-dev.py[99906] srx-07 commit complete /var/db/scripts/op/
  template/01-srx4600-local.j2 (md5: d31be14dadd2ce3437b2c1a3d84fd39d)

```

Then bulk configuration push is driven by profiles pre-defined in configuration file, to list profiles:

```

1 root@mx304-20-re0> op device-gen-auto-fbf list-profile all
2 -----
3 | # | push target profile |
4 -----
5 | 1 | srx4600_local |
6 -----
7 | 2 | srx4600_common |
8 -----
9 | 3 | srx4600_all |
10 -----

```

Example of push profile below defines name of bulk config push profile, devices where to template applies (maps to Netconf authentication profile not described here), device type (different Jinja2 template inputs), sequence (if applicable) and referral to Jinja2 template itself:

```

1 root@mx304-20-re0> op device-gen-auto-fbf show-profile srx4600_local
2 -----
3 | push target profile | device | device-type | sequence | template |
4 -----
5 | srx4600_local | srx-07 | srx4600 | 7 | 01-srx4600-local |
6 | | srx-08 | srx4600 | 8 | 01-srx4600-local |
7 | | srx-09 | srx4600 | 9 | 01-srx4600-local |
8 | | srx-10 | srx4600 | 10 | 01-srx4600-local |
9 -----

```

Note: it is perfectly possible to mix in the above profile different device types and templates, e.g., bulk push to mixed SRX4600/vSRX scale-out swarm.

Finally, result of bulk configuration push to multiple devices in a multi-threaded operation which takes in the specific setup less than 10 seconds:

```

1 root@mx304-20-re0> op device-gen-auto-fbf push-target-profile srx4600_local
2 -----
3 | device | template operation status message |
4 -----
5 | srx-07 | 01-srx4600-local template commit complete |
6 -----
7 | srx-08 | 01-srx4600-local template commit complete |
8 -----
9 | srx-09 | 01-srx4600-local template commit complete |
10 -----
11 | srx-10 | 01-srx4600-local template commit complete |
12 -----

```

Bulk changes to NAT pool size result – increase from /26 to /25 on remaining devices seen using Auto-FBF nat-info tool:

```

1 root@mx304-20-re0> op auto-fbf srx-nat-info all
2 -----
3 | instance | pool name | range | IPs | usage | peak usage | peak usage time | % failed session |
4 -----
5 | srx-07 | pool-1 | 3.0.7.0 - 3.0.7.127 | 128 | 36% | 42% | 2023-12-29 02:09:53 CET | 0.000 |
6 -----

```

7		srx-08		pool-1		3.0.8.0 - 3.0.8.127		128		37%		41%		2023-12-29 02:21:30 CET		0.000	
8	-----																
9		srx-09		pool-1		3.0.9.0 - 3.0.9.127		128		37%		42%		2023-12-29 02:21:30 CET		0.000	
10	-----																
11		srx-10		pool-1		3.0.10.0 - 3.0.10.127		128		36%		41%		2023-12-29 02:21:30 CET		0.000	
12	-----																

The tool also features optional archival of Jinja2 template and/or the pushed set commands upon commit, for both prototype device and bulk operations. Filename format is timestamp, template name, target device and appropriate suffix in specified folders which could be either the same or different for template and resulting set commands:

```

1 root@mx304-20-re0:/var/db/scripts/op/gen-dev-archive # ls -ltr
2 -rw-r--r-- 1 root wheel 13615 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-09.j2
3 -rw-r--r-- 1 root wheel 13540 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-09.set
4 -rw-r--r-- 1 root wheel 13615 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-10.j2
5 -rw-r--r-- 1 root wheel 13542 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-10.set
6 -rw-r--r-- 1 root wheel 13615 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-07.j2
7 -rw-r--r-- 1 root wheel 13535 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-07.set
8 -rw-r--r-- 1 root wheel 13615 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-08.j2
9 -rw-r--r-- 1 root wheel 13535 Jan 28 10:18 20240128-101827__tc-02-srx4600-all__srx-08.set

```

If both Jinja2 template and resulting set commands are archived, output of push-target-profile informs about the operation accordingly:

```

1 root@mx304-20-re0> op device-gen-auto-fbf push-target-profile srx4600_local
2 -----
3 | device | template operation status message |
4 -----
5 | srx-07 | 01-srx4600-local template commit complete, j2+set-cmd archived |
6 -----
7 | srx-08 | 01-srx4600-local template commit complete, j2+set-cmd archived |
8 -----
9 | srx-09 | 01-srx4600-local template commit complete, j2+set-cmd archived |
10 -----
11 | srx-10 | 01-srx4600-local template commit complete, j2+set-cmd archived |
12 -----

```

Then the applied templates and configuration can be tracked in time by MD5 checksum which is part of logging and available in template listing.

1.4 Troubleshooting

In case of issues like with template rendering, by default only high-level error message occurs:

```

1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local seq 7
2 -----
3 | device | template operation status message |
4 -----
5 | N/A | Error rendering template 01-srx4600-local, use debug on/see log |
6 -----

```

With debug flag on, output reveals in a verbose output misplaced “!” character on the 5th template line:

```

1 root@mx304-20-re0> op device-gen-auto-fbf device-type srx4600 template 01-srx4600-local seq 7
2 debug on
3 Traceback (most recent call last):
4   File "/var/db/scripts/op/auto-fbf-gen-dev.py", line 454, in template_thread
5     template = templateEnv.get_template(template_file)
6   File ".../src/dist/python-add-ons/jinja2/jinja2/environment.py", line 830,
7     in get_template
8   File ".../src/dist/python-add-ons/jinja2/jinja2/environment.py", line 804,
9     in _load_template
10  File ".../src/dist/python-add-ons/jinja2/jinja2/loaders.py", line 125, in
11  load

```

```

8 File ".../src/dist/python-add-ons/jinja2/jinja2/environment.py", line 591,
  in compile
9 File ".../src/dist/python-add-ons/jinja2/jinja2/environment.py", line 780,
  in handle_exception
10 File ".../src/dist/python-add-ons/jinja2/jinja2/_compat.py", line 37, in
   reraise
11 File "/var/db/scripts/op/template/01-srx4600-local.j2", line 12, in template
12 set groups auto-fbf security nat source pool pool-1 address 3.0.{{ seq }}.0/25
13 File ".../src/dist/python-add-ons/jinja2/jinja2/environment.py", line 497,
   in _parse
14 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 901, in
   parse
15 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 875, in
   subparse
16 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 620, in
   parse_tuple
17 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 432, in
   parse_expression
18 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 437, in
   parse_condexpr
19 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 450, in
   parse_or
20 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 459, in
   parse_and
21 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 470, in
   parse_not
22 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 474, in
   parse_compare
23 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 496, in
   parse_math1
24 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 507, in
   parse_concat
25 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 517, in
   parse_math2
26 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 528, in
   parse_pow
27 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 546, in
   parse_unary
28 File ".../src/dist/python-add-ons/jinja2/jinja2/parser.py", line 562, in
   parse_primary
29 File ".../src/dist/python-add-ons/jinja2/jinja2/lexer.py", line 359, in
   __next__
30 File ".../src/dist/python-add-ons/jinja2/jinja2/lexer.py", line 562, in
   wrap
31 File ".../src/dist/python-add-ons/jinja2/jinja2/lexer.py", line 683, in
   tokeniter
32 jinja2.exceptions.TemplateSyntaxError: unexpected '}'

```

1.5 Tool Help

As a last item, here is entire tool help screen when executed with no arguments (or incorrect arguments) with self-explanatory descriptions and recommended workflow described in this TechPost:

```

1 root@mx304-20-re0> op device-gen-auto-fbf
2
3 device-gen-auto-fbf is a simple yet powerfull tool for rendering and uploading Jinja2
4 templates,
5 designed for auto-fbf, but with general use-case in mind.
6 arguments:
7
8 device-type device type [vsrx|srx4600|mx|ptx]
9 seq device sequence number 1-n for template seeding
10 template name of j2 template
11 push-target SRX device name for template push
12 diff-target SRX device name to retrieve diff between candidate and running config
13 push-target-profile push to multiple SRX devices using profile
14 list-profile list push target profiles from auto_fbf_gen_dev_conf.py (any argument)
15 show-profile show details of push target profile [all|profile-name|# from list]
16 list-template list available Jinja2 template files (any argument)
17 show-template show contents of specific template [template-name|# from list ]
18 debug [on] enable verbose output related to template rendering
19
20 device-type, seq and template must be used together, push/diff-target is optional
21 push-target-profile is used as the only argument for bulk operation

```

```
22
23 The idea is to preview the template first in a form of set commands:
24
25 > op device-gen-auto-fbf device-type srx4600 template srx4600-local-01 seq 10
26
27 Then optionally diff the configuration against running config:
28
29 > op device-gen-auto-fbf device-type srx4600 template srx4600-local-01 seq 10 diff-target srx
30 -10
31
32 Finally push to single device and validate:
33
34 > op device-gen-auto-fbf device-type srx4600 template srx4600-local-01 seq 10 push-target srx
35 -10
36
37 where srx-10 is defined along with push-target-profile(s) below in auto_fbf_gen_dev_conf.py
38
39 Finally to upload proven template to multiple devices:
40
41 > op device-gen-auto-fbf push-target-profile p1
42
43 Templates are located in /var/db/scripts/op/template folder.
44
45 Both profiles and templates can be listed and viewed using corresponding list and show
  commands.
```

1.6 Conclusion

1.6.1 Availability

At the time of TechPost release (2024/02), the tool is not available publicly, feedback is still being collected within controlled group. Please reach out to the author in case of interest in test-driving (LinkedIn, feedback email).

1.7 Useful links

- Previous Auto-FBF TechPosts <https://juniper.github.io/techposts/scale-out-security-services-with-Auto-FBF/article> <https://juniper.github.io/techposts/operating-1tbps-firewall-scale-out-system/article>
- PyEZ developer guide <https://www.juniper.net/documentation/us/en/software/junos-pyez/junos-pyez-developer/index.html>
- Junos event options <https://www.juniper.net/documentation/us/en/software/junos/automation-scripting/topics/ref/statement/event-options-edit.html>
- Ephemeral Configuration Database <https://www.juniper.net/documentation/us/en/software/junos/junos-xml-protocol/topics/concept/ephemeral-configuration-database-overview.html>

1.8 Glossary

- ASN: Autonomous System Number
- APN: Access Point Name
- BGP: Border Gateway Protocol
- CLI: Command Line Interface
- FBF: Firewall-Based Forwarding
- NAT: Network Address Translation

1.9 Acknowledgments

Juniper account teams pushing the envelope and supporting related activities. Then all the people I have the pleasure to work with - my manager Dirk Van den Borne, colleagues Steven Jacques, Mark Barrett, Pawel Rabiej, Javier Grizzuti, Dezso Csonka, Theodore Jenks, Akram Hentati, Matthijs Nagel and the entire Amsterdam POC crew providing equipment and support.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

